

Unit-5: Relational Database design [Also Refer PPT]

GTU Syllabus:

Relational Database Design:

Features of good relational designs, Dependency preservation, Functional Dependency–definition, Trivial and Non-Trivial FD, Closure of FD set, Closure of attributes, Irreducible set of FD, Decomposition using FD, Normalization – 1NF, 2NF, 3NF,BCNF, Multivalued dependency, 4NF.

Features of good relational designs:

A good relational design features minimal data redundancy, no modification anomalies (preventing data loss or corruption during updates, deletions, or insertions), lossless decomposition, and clear, modular tables. It ensures data consistency, integrity, and accuracy, making it efficient for data fetching and management while remaining flexible for future expansion.

Key Characteristics of Good Relational Design

➤ **No Data Redundancy:**

Data should be stored only once to save space and prevent inconsistencies.

➤ **No Modification Anomalies:**

The design must prevent issues like update anomalies (updating the same data multiple times), insertion anomalies (inability to add new data without other related data), and deletion anomalies (losing unrelated data when deleting something).

➤ **Lossless Decomposition:**

When breaking down larger schemas into smaller, more manageable ones, no information should be lost in the process.

➤ **Single Entity/Relationship per Relation:**

Each table (relation) should focus on a single, clearly defined entity or relationship to maintain modularity.

➤ **Minimal Null Values:**

Avoid using null values unnecessarily, as they can complicate data retrieval and database operations.

➤ **Data Consistency:**

The design should ensure that data across different parts of the database remains consistent and accurate.

➤ **Data Integrity:**

It supports and maintains the accuracy and reliability of the data.

➤ **Flexibility and Modularity:**

A good design allows for future expansion and modifications without requiring major structural changes.

➤ **Efficient Data Fetching:**

The structure of the design should support efficient retrieval and usage of data within the application.

➤ **Adherence to Normal Forms:**

Through normalization, the design adheres to rules like the First Normal Form (1NF) and Boyce-Codd Normal Form(BCNF) to achieve these goals.

Functional Dependency in DBMS:

In relational database management, functional dependency is a concept that specifies the relationship between two sets of attributes where one attribute determines the value of another attribute. It is denoted as $X \rightarrow Y$, where the attribute set on the left side of the arrow, X is called Determinant, and Y is called the Dependent.

What is Functional Dependency?

A functional dependency occurs when one attribute uniquely determines another attribute within a relation. It is a constraint that describes how attributes in a table relate to each other. If attribute A functionally determines attribute B we write this as the $A \rightarrow B$.

Functional dependencies are used to mathematically express relations among database entities and are very important to understanding advanced concepts in Relational Database Systems.

Example:

roll_no	name	dept_name	dept_building
---------	------	-----------	---------------

42	abc	CO	A4
43	pqr	IT	A3
44	xyz	CO	A4
45	xyz	IT	A3
46	mno	EC	B2
47	jkl	ME	B2

From the above table we can conclude some valid functional dependencies:

- $\text{roll_no} \rightarrow \{ \text{name}, \text{dept_name}, \text{dept_building} \}$ → Here, roll_no can determine values of fields name, dept_name and dept_building, hence a valid Functional dependency
- $\text{roll_no} \rightarrow \text{dept_name}$, Since, roll_no can determine whole set of {name, dept_name, dept_building}, it can determine its subset dept_name also.
- $\text{dept_name} \rightarrow \text{dept_building}$, Dept_name can identify the dept_building accurately, since departments with different dept_name will also have a different dept_building
- More valid functional dependencies: $\text{roll_no} \rightarrow \text{name}$, $\{ \text{roll_no}, \text{name} \} \twoheadrightarrow \{ \text{dept_name}, \text{dept_building} \}$, etc.

Here are some invalid functional dependencies:

- $\text{name} \rightarrow \text{dept_name}$ Students with the same name can have different dept_name, hence this is not a valid functional dependency.
- $\text{dept_building} \rightarrow \text{dept_name}$ There can be multiple departments in the same building. Example, in the above table departments ME and EC are in the same building B2, hence $\text{dept_building} \rightarrow \text{dept_name}$ is an invalid functional dependency.

- More invalid functional dependencies: $\text{name} \rightarrow \text{roll_no}$, $\{\text{name}, \text{dept_name}\} \rightarrow \text{roll_no}$, $\text{dept_building} \rightarrow \text{roll_no}$, etc.

Types of Functional Dependencies in DBMS

1. Trivial functional dependency
2. Non-Trivial functional dependency
3. Multivalued functional dependency
4. Transitive functional dependency

1. Trivial Functional Dependency

In Trivial Functional Dependency, a dependent is always a subset of the determinant. i.e. If $X \rightarrow Y$ and Y is the subset of X , then it is called trivial functional dependency.

Symbolically: $A \rightarrow B$ is trivial functional dependency if B is a subset of A .

The following dependencies are also trivial: $A \rightarrow A$ & $B \rightarrow B$

Example 1 :

- $ABC \rightarrow AB$
- $ABC \rightarrow A$
- $ABC \rightarrow ABC$

Example 2:

roll_no	name	age
42	abc	17

43	pqr	18
44	xyz	18

Here, $\{\text{roll_no}, \text{name}\} \rightarrow \text{name}$ is a trivial functional dependency, since the dependent name is a subset of determinant set $\{\text{roll_no}, \text{name}\}$. Similarly, $\text{roll_no} \rightarrow \text{roll_no}$ is also an example of trivial functional dependency.

2. Non-trivial Functional Dependency

In Non-trivial functional dependency, the dependent is strictly not a subset of the determinant. i.e. If $X \rightarrow Y$ and Y is not a subset of X, then it is called Non-trivial functional dependency.

Example 1 :

- Id -> Name
- Name -> DOB

Example 2:

roll_no	name	age
42	abc	17
43	pqr	18
44	xyz	18

Here, $\text{roll_no} \rightarrow \text{name}$ is a non-trivial functional dependency, since the dependent name is not a subset of determinant roll_no. Similarly, $\{\text{roll_no}, \text{name}\} \rightarrow \text{age}$ is also a non-trivial functional dependency, since age is not a subset of $\{\text{roll_no}, \text{name}\}$

3. Semi Non Trivial Functional Dependencies

A semi non-trivial functional dependency occurs when part of the dependent attribute (right-hand side) is included in the determinant (left-hand side), but not all of it. This is a middle ground between trivial and non-trivial functional dependencies. $X \rightarrow Y$ is called semi non-trivial when $X \cap Y$ is not NULL.

Example:

Consider the following table:

Student_ID	Course_ID	Course_Name
101	CSE101	Computer Science
102	CSE102	Data Structures
103	CSE101	Computer Science

Functional Dependency:

$\{\text{StudentID}, \text{CourseID}\} \rightarrow \text{CourseID}$

This is semi non-trivial because:

- Part of the dependent attribute (Course_ID) is already included in the determinant ($\{\text{Student_ID}, \text{Course_ID}\}$).
- However, the dependency is not completely trivial because $\{\text{StudentID}\} \rightarrow \text{CourseID}$ is not implied directly.

4. Multivalued Functional Dependency

In Multivalued functional dependency, entities of the dependent set are not dependent on each other. i.e. If $a \twoheadrightarrow \{b, c\}$ and there exists no functional dependency between b and c, then it is called a multivalued functional dependency.

Example:

bike_model	manuf_year	color
tu1001	2007	Black
tu1001	2007	Red
tu2012	2008	Black
tu2012	2008	Red
tu2222	2009	Black
tu2222	2009	Red

In this table:

- X: bike_model
- Y: color
- Z: manuf_year

For each bike model (bike_model):

1. There is a group of colors (color) and a group of manufacturing years (manuf_year).
2. The colors do not depend on the manufacturing year, and the manufacturing year does not depend on the colors. They are independent.
3. The sets of color and manuf_year are linked only to bike_model.

That's what makes it a multivalued dependency.

In this case these two columns are said to be multivalued dependent on bike_model. These dependencies can be represented like this:

5. Transitive Functional Dependency

In transitive functional dependency, dependent is indirectly dependent on determinant. i.e. If $a \rightarrow b$ & $b \rightarrow c$, then according to axiom of transitivity, $a \rightarrow c$. This is a transitive functional dependency.

Example:

enrol_no	name	dept	building_no
42	abc	CO	4
43	pqr	EC	2
44	xyz	IT	1
45	abc	EC	2

Here, $\text{enrol_no} \rightarrow \text{dept}$ and $\text{dept} \rightarrow \text{building_no}$. Hence, according to the axiom of transitivity, $\text{enrol_no} \rightarrow \text{building_no}$ is a valid functional dependency. This is an indirect functional dependency, hence called Transitive functional dependency.

6. Fully Functional Dependency

In full functional dependency an attribute or a set of attributes uniquely determines another attribute or set of attributes. If a relation R has attributes X, Y, Z with the dependencies $X \rightarrow Y$ and $X \rightarrow Z$ which states that those dependencies are fully functional.

7. Partial Functional Dependency

In partial functional dependency a non key attribute depends on a part of the composite key, rather than the whole key. If a relation R has attributes X, Y, Z where X and Y are the composite key and Z is non key attribute. Then $X \rightarrow Z$ is a partial functional dependency in RBDMS.

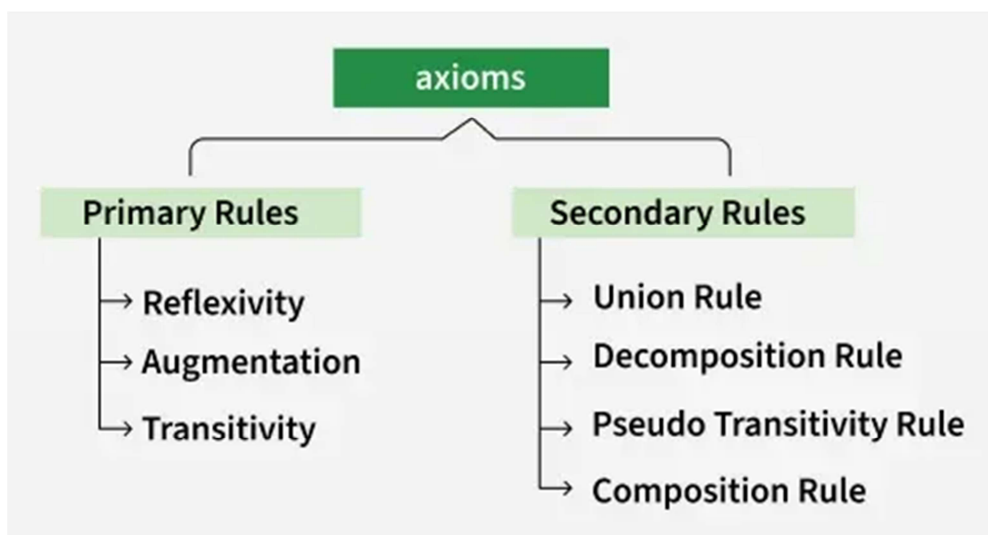
Advantages of Functional Dependency

The advantages of functional dependency are many. Some of them are listed below:

- The database's data quality is maintained using it.
- It communicates the database design's facts.
- It aids in precisely outlining the limitations and implications of databases.
- It is useful to recognize poor designs.
- Functional Dependency eliminates data duplication in cases where the same values shouldn't appear more than once in the same database table.
- Finding the potential keys in the relationship is the first step in the **normalization** procedure. Identifying potential keys and normalizing the database without functional dependencies is impossible.

Armstrong Axioms [IMP]

- The term Armstrong Axioms refers to the sound and complete set of inference rules or axioms, introduced by William W. Armstrong that is used to test the logical implication of **functional dependencies**.
- If F is a set of functional dependencies then the closure of F, denoted as F^+ , is the set of all functional dependencies logically implied by F. Armstrong's Axioms are a set of rules, that when applied repeatedly, generates a closure of functional dependencies.



Axioms

- **Axiom of Reflexivity:** If A is a set of attributes and B is a subset of A, then A holds B. If $B \subseteq A$ then $A \rightarrow B$. This property is trivial property.

- **Axiom of Augmentation:** If $A \rightarrow B$ holds and Y is the attribute set, then $AY \rightarrow BY$ also holds. That is adding attributes to dependencies, does not change the basic dependencies. If $A \rightarrow B$, then $AC \rightarrow BC$ for any C .
- **Axiom of Transitivity:** Same as the transitive rule in algebra, if $A \rightarrow B$ holds and $B \rightarrow C$ holds, then $A \rightarrow C$ also holds. $A \rightarrow B$ is called A functionally which determines B . If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$.

Secondary Rules

These rules can be derived from the above axioms.

- **Union:** If $A \rightarrow B$ holds and $A \rightarrow C$ holds, then $A \rightarrow BC$ holds. If $X \rightarrow Y$ and $X \rightarrow Z$ then $X \rightarrow YZ$.
- **Composition:** If $A \rightarrow B$ and $X \rightarrow Y$ hold, then $AX \rightarrow BY$ holds.
- **Decomposition:** If $A \rightarrow BC$ holds then $A \rightarrow B$ and $A \rightarrow C$ hold. If $X \rightarrow YZ$ then $X \rightarrow Y$ and $X \rightarrow Z$.
- **Pseudo Transitivity:** If $A \rightarrow B$ holds and $BC \rightarrow D$ holds, then $AC \rightarrow D$ holds. If $X \rightarrow Y$ and $YZ \rightarrow W$ then $XZ \rightarrow W$.
- **Self Determination:** It is similar to the Axiom of Reflexivity, i.e. $A \rightarrow A$ for any A .
- **Extensivity:** Extensivity is a case of augmentation. If $AC \rightarrow A$, and $A \rightarrow B$, then $AC \rightarrow B$. Similarly, $AC \rightarrow ABC$ and $ABC \rightarrow BC$. This leads to $AC \rightarrow BC$.

Armstrong Relation

Armstrong Relation can be stated as a relation that is able to satisfy all functional dependencies in the F+ Closure. In the given set of dependencies, the size of the minimum Armstrong Relation is an exponential function of the number of attributes present in the dependency under consideration.

Why Armstrong Axioms Refer to the Sound and Complete?

By sound, we mean that given a set of functional dependencies F specified on a relation schema R , any dependency that we can infer from F by using the primary rules of Armstrong axioms holds in every relation state r of R that satisfies the dependencies in F . By complete, we mean that using primary rules of Armstrong axioms repeatedly to infer dependencies until no more dependencies can be inferred results in the complete set of all possible dependencies that can be inferred from F .

Advantages of Using Armstrong's Axioms in Functional Dependency

- They provide a systematic and efficient method for inferring additional functional dependencies from a given set of functional dependencies, which can help to optimize database design.
- They can be used to identify redundant functional dependencies, which can help to eliminate unnecessary data and improve database performance.
- They can be used to verify whether a set of functional dependencies is a minimal cover, which is a set of dependencies that cannot be further reduced without losing information.

Disadvantages of Using Armstrong's Axioms in Functional Dependency

- The process of using Armstrong's axioms to infer additional functional dependencies can be computationally expensive, especially for large databases with many tables and relationships.
- The axioms do not take into account the semantic meaning of data, and may not always accurately reflect the relationships between data elements.
- The axioms can result in a large number of inferred functional dependencies, which can be difficult to manage and maintain over time.

Attribute Closure

Attribute Closure of an attribute set is defined as a set of all attributes that can be functionally determined from it.

The closure of an attribute is represented as $^+$

Finding Closure of an attribute set

You can follow the steps to find the Closure of an attribute set:

1. Determine A^+ , the Closure of A under functional dependency set F.
2. A^+ will contain A itself; **For example**, if we need to find the closure of an attribute X, the closure will incorporate the X itself and the other attributes that the X attribute can determine.
3. Repeat the process as
4. old $A^+ = A$ Closure;
5. for each FD $X \rightarrow Y$ in the FD set, do
6. if X Closure is a subset of X, then $A \text{ Closure} := A \text{ Closure} \cup Y$;
7. Repeat until ($A^+ = \text{old } A^+$);

For example,

Given a relation $R(A,B,C,D)$ and FD $\{A \rightarrow B, B \rightarrow C, C \rightarrow D\}$, then determine the A^+ :

1. $A^+ = A$, since A can determine A itself.
2. $A^+ = AB$, A can also determine B, it is because in the FD set $A \rightarrow B$ dependency is given.
3. $A^+ = ABC$, A can also determine C with the help of B, since $A \rightarrow B, B \rightarrow C$, **thus, $A \rightarrow C$** // Transitive property
4. $A^+ = ABCD$, A can also determine D with the help of the C attribute, since C is already determined now in the FD set functional dependency $C \rightarrow D$ holds, D can be determined with C.

Therefore,

$A^+(A \text{ closure}) = ABCD$. A can determine all attributes (ABCD).

Now let's see some questions for a clear understanding of the concept.

Question: In a schema with attributes A, B, C, D, E following set of functional dependencies are given

$\{A \rightarrow B, A \rightarrow C, CD \rightarrow E, B \rightarrow D, E \rightarrow A\}$,

Which of the following dependencies is **not** implied by the above set? (GATE CS 2005).

1. $CD \rightarrow AC$
2. $BD \rightarrow CD$
3. $BC \rightarrow CD$
4. $AC \rightarrow BC$

Solution:

Checking option A, $CD \rightarrow AC$,

1. CD can determine C and D (can determine themselves) and E (as $CD \rightarrow E$, given in FD set). Therefore, $(CD)^+ = CDE$
2. Now E can determine A (as $E \rightarrow A$), $(CD)^+ = CDEA$
3. With the help of A, we can determine B, $(CD)^+ = CDEAB$

So, $(CD)^+ = ABCDE$

We can clearly see that with CD, AC can be determined. Therefore, $CD \rightarrow AC$ holds true.

Checking for Option B, $BD \rightarrow CD$,

On taking BD closure $(BD)^+ = BD$ as they can determine themselves only, no other dependencies in the FD set.

We can see that $BD \rightarrow CD$ does not hold; **hence B is the correct option.**

Other options can be checked similarly by taking closure.

Another example:

Question: The following functional dependencies are given:

$\{AB \rightarrow CD, AF \rightarrow D, DE \rightarrow F, C \rightarrow G, F \rightarrow E, G \rightarrow A\}$

Which one of the following options is false? (GATE 2006)

1. $CF^+ = \{ACDEFG\}$
2. $BG^+ = \{ABCDG\}$
3. $AF^+ = \{ACDEFG\}$
4. $AB^+ = \{ABCDG\}$

Solution:

If we take the attribute closure of option A, we will get, $(CF)^+ = \{ACDEFG\}$

If we take the attribute closure of option B, we will get, $(BG)^+ = \{ABCDG\}$

This can be done with the steps discussed above in the article.

But option C and D have attribute closure: $(AF)^+ = \{AFDE\}$ and $(AB)^+ = \{ABCDG\}$.

Therefore, options C and D are false.

Finding Candidate Keys and Super Keys using the Attribute Closure

A candidate key is the minimal set of attributes that can uniquely identify a tuple. For example, STU_ID in the 'Student' relation can be a candidate key. The candidate key should be unique and not null.

A superkey is also a set of attributes that can uniquely identify a tuple in a given relation. The concept of the candidate key and the super key is closely related. Super key reduced to the minimum number of attributes makes the candidate key, which means the super key is the superset of a candidate key. In a given relation, 'Student' STU_ID and STU_NAME can be super key.

The attribute closure method can also be used to find all the **candidate keys and super keys** in the given relation.

If the attribute closure of an attribute set contains all the attributes of the given relation, then the attribute set will be the **superkey** of the relation.

Similarly, if no subset of this attribute set can functionally determine all relation attributes, the present set will be a candidate key.

For example given a relation $R(A,B,C,D)$ and FD $\{A \rightarrow B, B \rightarrow C, C \rightarrow D, D \rightarrow A\}$, then

$A^+ = BCDA$,

$B^+ = ACDB$

$C^+ = ACDB$

$D^+ = ACDB$

Since all A, B, C, D can functionally determine all the attributes, so candidate key set will be $\{A, B, C, D\}$. With the help of this, we can also determine prime and non-prime attributes.

Prime attributes are the set of all attributes present in the candidate key, and non-prime attributes are the remaining attributes of the relation.

So here, Prime attributes are $\{A, B, C, D\}$ and there is no non-prime attribute(non-prime attribute set will be empty $\{\}$).

Superkeys can be formed by combining other attributes with the candidate key. Suppose AB can uniquely identify all the other attributes in the given, and A alone can do this, so AB be super key and A will be candidate key

GATE Question: Consider a relation scheme $R = (A, B, C, D, E, H)$ on which the following functional dependencies hold: $\{A \rightarrow B, BC \rightarrow D, E \rightarrow C, D \rightarrow A\}$. What are the candidate keys of R?

1. AE, BE
2. AE, BE, DE
3. AEH, BEH, BCH
4. AEH, BEH, DEH

Solution: A set of attributes S is a candidate key of relation R if the closure of S is all attributes of R and there is no subset of S whose closure is all attributes of R.

If we look closely, attributes E and H are not determined by any of the dependencies given in the FD set. Therefore they need to be present on the left-hand side. Only option D satisfies the given condition. Therefore, we can check for attribute closure.

$(AEH)^+ = \{ABCDEH\}$

$(BEH)^+ = \{ABCDEH\}$

$(DEH)^+ = \{ABCDEH\}$

Option D is correct.

Decomposition using FD:

Decomposition in the context of database design refers to the process of breaking down a single table into multiple tables in order to eliminate redundancy, reduce data anomalies, and achieve normalization. Decomposition is typically done using rules defined by normalization forms.

However, while decomposition can be helpful, it is not without challenges. Done incorrectly, decomposition can lead to its own set of problems.

Problems Related to Decomposition:

1. Loss of Information

- **Non-loss decomposition:** When a relation is decomposed into two or more smaller relations, and the original relation can be perfectly reconstructed by taking the natural

join of the decomposed relations, then it is termed as **lossless decomposition**. If not, it is termed "**lossy decomposition**."

- Example: Let's consider a table $R(A, B, C)$ with a dependency $A \rightarrow B$. If you decompose it into $R_1(A, B)$ and $R_2(B, C)$, it would be lossy because you can't recreate the original table using natural joins.

Example: Consider a relation $R(A,B,C)$ with the following data:

A	B	C
1	X	P
1	Y	P
2	Z	Q

Suppose we decompose R into $R_1(A,B)$ and $R_2(A,C)$.

$R_1(A, B)$:

A	B
1	X
1	Y
2	Z

$R_2(A, C)$:

A	C
1	P
1	P
2	Q

Now, if we take the natural join of R_1 and R_2 on attribute A , we get back the original relation R . Therefore, this is a lossless decomposition.

2. Loss of Functional Dependency

- Once tables are decomposed, certain functional dependencies might not be preserved, which can lead to the inability to enforce specific integrity constraints.
- Example: If you have the functional dependency $A \rightarrow B$ in the original table, but in the decomposed tables, there is no table with both A and B , this functional dependency can't be preserved.

Example: Let's consider a relation R with attributes A, B , and C and the following functional dependencies:

$A \rightarrow B$

$B \rightarrow C$

Now, suppose we decompose R into two relations:

R1(A,B) with FD $A \rightarrow B$

R2(B,C) with FD $B \rightarrow C$

In this case, the decomposition is dependency-preserving because all the functional dependencies of the original relation R can be found in the decomposed relations R1 and R2. We do not need to join R1 and R2 to enforce or check any of the functional dependencies.

However, if we had a functional dependency in R, say $A \rightarrow C$, which cannot be determined from either R1 or R2 without joining them, then the decomposition would not be dependency-preserving for that specific FD.

3. Increased Complexity

- Decomposition leads to an increase in the number of tables, which can complicate queries and maintenance tasks. While tools and ORM (Object-Relational Mapping) libraries can mitigate this to some extent, it still adds complexity.

4. Redundancy

- Incorrect decomposition might not eliminate redundancy, and in some cases, can even introduce new redundancies.

5. Performance Overhead

- An increased number of tables, while aiding normalization, can also lead to more complex SQL queries involving multiple joins, which can introduce performance overheads.

Normalization and its Types [You can also refer from PPT]

Normalization is a process of organizing data in a database to reduce redundancy and improve data consistency. Primary keys are really important in organizing information in a database. They help to make sure that every row in a table has a unique identification so that nothing gets mixed up or lost.

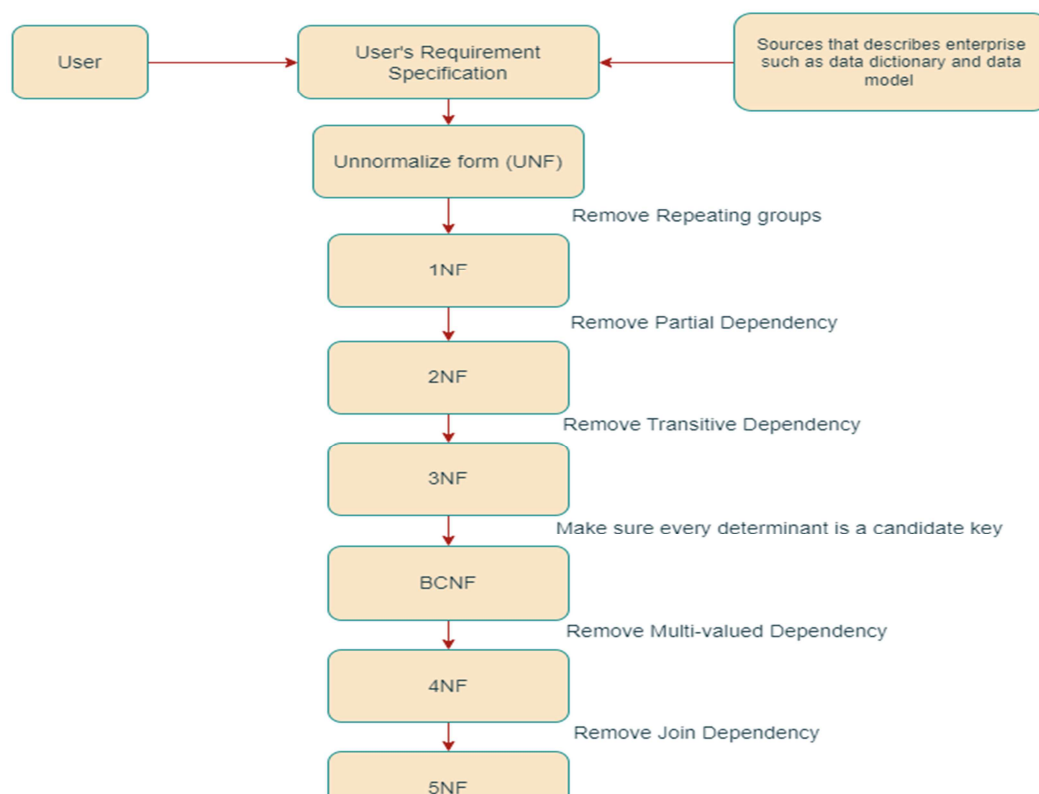
Normalization is a process of organizing the data in database to avoid data redundancy, insertion anomaly, update anomaly & deletion anomaly.

Why do we need Normalization?

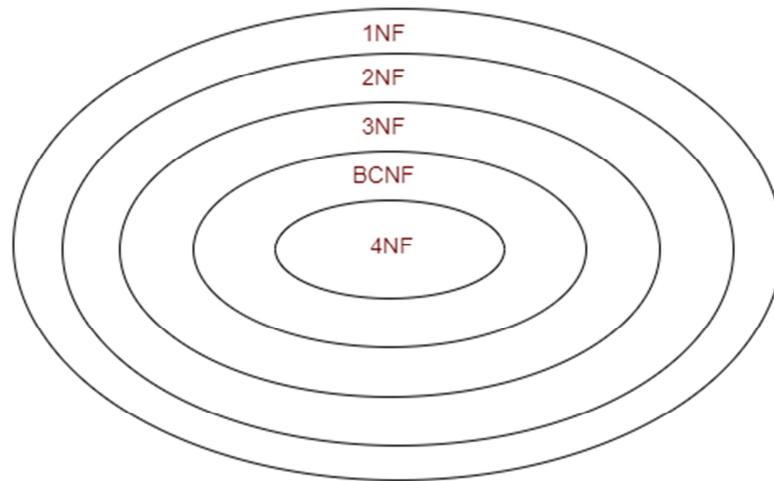
The main reason for normalizing the relations is removing these anomalies. Failure to eliminate anomalies leads to data redundancy and can cause data integrity and other problems as the database grows. Normalization consists of a series of guidelines that helps to guide you in creating a good database structure.

Data modification anomalies can be categorized into three types:

- **Insertion Anomaly:** Insertion Anomaly refers to when one cannot insert a new tuple into a relationship due to lack of data.
- **Deletion Anomaly:** The delete anomaly refers to the situation where the deletion of data results in the unintended loss of some other important data.
- **Updation Anomaly:** The update anomaly is when an update of a single data value requires multiple rows of data to be updated.



Normalization is the process of organizing data in a database to minimize redundancy and dependency. In database design, there are different normal forms based on the primary keys of a table. These include –



First Normal Form (1NF)

1NF requires that each column in a table contains atomic values and that each row is uniquely identified. This means that a table cannot have repeating groups or arrays as columns, and each row must have a unique primary key.

Example

A table is in 1NF if each column contains atomic values and each row is uniquely identified. For example, a table that lists customers and their phone numbers –

Customer ID	Name	Phone Numbers
1	John	555-1234, 555-5678
2	Jane	555-9876
3	Michael	555-5555

This violates 1NF because the Phone Numbers column contains repeating groups.

To normalize this table to 1NF, we can split the Phone Numbers column into separate rows and add a separate primary key column –

Customer ID	Name	Phone Number
1	John	555-1234

Customer ID	Customer Name
-------------	---------------

1	John	555-5678
2	Jane	555-9876
3	Michael	555-5555

Second Normal Form (2NF)

2NF builds on 1NF by requiring that each non-primary key column in a table is fully functionally dependent on the primary key. This means that a table should not have partial dependencies, where a non-primary key column depends on only part of the primary key.

Example

A table is in 2NF if each non-primary key column is fully functionally dependent on the primary key. For example, a table that lists orders and their line items:

Order ID	Customer ID	Customer Name	Item ID	Item Name	Quantity
1	1	John	1	Shirt	2
1	1	John	2	Pants	1
2	2	Jane	1	Shirt	1
2	2	Jane	3	Hat	3

This violates 2NF because the Customer Name column depends on only part of the primary key (Customer ID). To normalize this table to 2NF, we can split it into two tables –

Order ID	Customer ID	Item ID	Quantity
1	1	1	2
1	1	2	1
2	2	1	1
2	2	3	3

1	John
2	Jane

Third Normal Form (3NF)

3NF builds on 2NF by requiring that each non-primary key column in a table is not transitively dependent on the primary key. This means that a table should not have transitive dependencies, where a non-primary key column depends on another non-primary key column.

Example

To explain 3NF further, let's consider an example of a table that lists customer orders –

Order ID	Customer ID	Customer Name	Customer City	Order Date	Order Total
1	100	John Smith	New York	2022-01-01	100
2	101	Jane Doe	Los Angeles	2022-01-02	200
3	102	Bob Johnson	San Francisco	2022-01-03	300

In this example, the non-primary key column "Customer City" is transitively dependent on the primary key. That is, it depends on "Customer ID", which is not part of the primary key, instead of depending directly on the primary key "Order ID". To bring this table to 3NF, we can split it into two tables –

Table 1: Customers

Customer ID	Customer Name	Customer City
100	John Smith	New York
101	Jane Doe	Los Angeles
102	Bob Johnson	San Francisco

Table 2: Orders

Order ID	Customer ID	Order Date	Order Total
1	100	2022-01-01	100
2	101	2022-01-02	200
3	102	2022-01-03	300

Now, the "Customer City" column is no longer transitively dependent on the primary key and is instead in a separate table that has a direct relationship with the primary key. This makes the table 3NF-compliant.

Boyce-Codd Normal Form (BCNF)

BCNF is a stricter form of 3NF that applies to tables with more than one candidate key. BCNF requires that each non-trivial dependency in a table is a dependency on a candidate key. This means that a table should not have non-trivial dependencies, where a non-primary key column depends on another non-primary key column. BCNF ensures that each table in a database is a separate entity and eliminates redundancies.

Example

A table is in BCNF if each determinant is a candidate key. In other words, every non-trivial functional dependency in the table must be on a candidate key. For example, consider a table that lists information about books and their authors –

Table: Books

Book ID	Title	Author ID	Author Name	Author Nationality
1	Crime and Punishment	100	Fyodor Dostoevsky	Russian
2	The Great Gatsby	101	F. Scott Fitzgerald	American
3	Pride and Prejudice	102	Jane Austen	British

In this example, the functional dependency between "Author ID" and "Author Name" violates BCNF because it is not on a candidate key. To bring this table to BCNF, we can split it into two tables –

Table 1: Authors

Author ID	Author Name	Author Nationality
101	Fyodor Dostoevsky	Russian
101	F. Scott Fitzgerald	American
102	Jane Austen	British

Table 2: Books

Book ID	Title	Author ID
1	Crime and Punishment	100
2	The Great Gatsby	101
3	Pride and Prejudice	102

Now, the "Author Name" and "Author Nationality" columns are not transitively dependent on the primary key, and the table is in BCNF.

Fourth Normal Form (4NF)

4NF builds on BCNF by requiring that a table should not have multi-valued dependencies. A multi-valued dependency occurs when a non-primary key column depends on a combination of other non-primary key columns. For example, a table that lists customer orders with a primary key of order ID and non-primary key columns for customer ID and order items violates 4NF because order items depend on both order ID and customer ID.

For example, a table that lists orders and their products, with columns for order ID, product ID, and product details, violates 4NF because the product details depend on the combination of order ID and product ID.

Example

Consider the following table of orders and products

Order ID	Product ID	Product Name	Product Description
----------	------------	--------------	---------------------

1	100	Widget	Red Widget
1	200	Widget	Blue Widget
2	100	Widget	Red Widget
2	300	Thing	Green Thing
3	200	Widget	Blue Widget
3	300	Thing	Green Thing

In this table, the product name and description depend on both the order ID and product ID, creating a multi-valued dependency. To bring the table into 4NF, we can split it into three tables –

Order ID		Product ID	
1		100	
1		200	
2		100	
2		300	
3		200	
3		300	
Product ID		Product Name	
100		Widget	
200		Widget	
300		Thing	
Product ID		Product Description	
100		Red Widget	
200		Blue Widget	

Advantages and Disadvantages of Normalization:**Advantages of Normalization**

- Normalization helps to minimize data redundancy.
- Greater overall database organization.
- Data consistency within the database.
- Much more flexible database design.
- Enforces the concept of relational integrity.

Disadvantages of Normalization

- You cannot start building the database before knowing what the user needs.
- The performance degrades when normalizing the relations to higher normal forms, i.e., 4NF, 5NF.
- It is very time-consuming and difficult to normalize relations of a higher degree.
- Careless decomposition may lead to a bad database design, leading to serious problems.

How to Find Candidate Key from Functional Dependencies**Functional Dependency**

- Functional dependency is the relations between two sets of attributes X and Y such that if $X \rightarrow Y$, then all the same values of X there will be the same values of Y, or Y can be identified by the values of X uniquely.

Candidate Key

- In a relational model, the set of those attributes which can be used to identify each row uniquely is called super keys, and there can be a lot of super keys. Those super keys which have no proper subset as any super key then it is called a candidate key.

For example, in a relational model, we have three attributes as {A, B, C} and {AB} can identify a complete table, and {A} itself can identify the whole table then AB and A both will be super key, but A will be candidate key.

Closure

Attribute closure of an attribute can be identified as the set of attributes that can be functionally determined from it.

Closures will be used to find out the candidate key using functional dependencies.

Steps to find out the Candidate Key of a Relational Table using Functional Dependencies

Step 1:

First, we will find out the essential and non-essential sets of attributes from the given set of attributes. Those attributes which are dependent on other attributes are non-essential attributes, and their values can be found out using essential attributes.

So, all the essential attributes will definitely be part of our candidate key.

Step 2:

We will combine all the essential attributes, and if they can determine all the attributes (by finding their closure), then it will be the candidate key.

Step 3:

If a combination of essential attributes is not a candidate key, then we will try different combinations of essential and non-essential attributes which are not subset of each other to find out all the candidate keys.

Example 1:

Let's suppose we have a set of attributes as S: {A, B, C, D} and functional dependencies are:

A → B

B → C

C → A

Solution:

In the above example, we have non-essential attributes as {A, B, C}, and the essential attribute is {D}.

So, D will be the part of the candidate key.

Closure of D: {D}

So, a combination of essential attributes is not able to find all the attributes, so we will add non-essential attributes in different ways to find out the candidate keys.

Closure of (AD) = {A, B, C, D} (using $A \rightarrow B$ and $B \rightarrow C$)

So, AD will be a candidate key.

Closure of (BD) = {A, B, C, D} (using $C \rightarrow A$ and $B \rightarrow C$)

So, BD will be a candidate key.

Closure of (CD) = {A, B, C, D} (using $C \rightarrow A$ and $A \rightarrow B$)

So, CD will be a candidate key.

No other combination of attributes is not possible, so candidate keys are {AD, BD, CD}

Example 2:

Let's suppose we have a set of attributes as S: {A, B, C, D, E, F} and functional dependencies are:

$AB \rightarrow C$

$B \rightarrow AE$

$C \rightarrow D$

Solution:

In the above example, we have non-essential attributes as {A, C, D, E}, and the essential attribute is {B, F}.

So, BF will be the part of the candidate key.

Closure of BF: {B, A, E, C, D, F}

Since the combination of essential attributes is capable of finding all the attributes, it will be a candidate key. All other combinations of the essential and non-essential attributes will not be minimal, so there will be only one candidate key as {BF}.

Example 3:

Let's suppose we have a set of attributes as S: {A, B, C, D, E} and functional dependencies are:

$CE \rightarrow D$

$D \rightarrow B$

$C \rightarrow A$

Solution:

In the above example, we have non-essential attributes as {A, B, D}, and the essential attribute is {C, E}.

So, CE will be the part of the candidate key.

Closure of CE: {C,E,D,B,A} using (CE $\rightarrow$ D, D $\rightarrow$ B and C $\rightarrow$ A)

Since the combination of essential attributes is capable of finding all the attributes, it will be a candidate key. All other combinations of the essential and non-essential attributes will not be minimal, so there will be only one candidate key as {CE}.

Example 4:

Let's suppose we have a set of attributes as S: {W, X, Y, Z} and functional dependencies are:

$Z \rightarrow W$

$Y \rightarrow XZ$

$XW \rightarrow Y$

Solution:

In the above example, we have non-essential attributes as {W, X, Y, Z}, and the essential attribute is {}.

In this example, we will explore each and every combination of attributes to find out the candidate key, as there is no essential attribute.

Closure of W = {W}

Closure of X = {X}

Closure of Y = {Y,X,Z,W} (using Y $\rightarrow$ XZ and Z $\rightarrow$ W)

Closure of Z = {Z}

Since Y is capable of finding all the attributes so it will be a candidate key. Y will not be part of the next combinations.

Closure of XW = {X,W,Y,Z} (using XW $\rightarrow$ Y and Y $\rightarrow$ XZ)

Closure of ZW = {Z,W}

Closure of XZ = {X,W,Y,Z} (using $Z \rightarrow W$ and $XW \rightarrow Y$)

So XW and XZ will be the next candidate keys, and the next other combinations will contain no attributes apart from these candidate keys.

So there will be three candidate keys which are {Y, XW, XZ}

Example 5:

Let's suppose we have a set of attributes as S: {A, B, C, D, E, F} and functional dependencies are:

$AB \rightarrow C$

$C \rightarrow D$

$D \rightarrow BE$

$E \rightarrow F$

$F \rightarrow A$

Solution:

In the above example, we have non-essential attributes as {A, B, C, D, E, F}, and the essential attribute is {}.

Now in this example, we will explore each and every combination of attributes to find out the candidate key, as there is no essential attribute.

If there is no essential attribute, then each non-essential attribute can be a candidate key, so it increases the number of candidate keys.

Closure of A = {A}

Closure of B = {B}

Closure of C = {C, D, B, E, F, A} using ($C \rightarrow D$, $D \rightarrow BE$, $E \rightarrow F$, $F \rightarrow A$)

Closure of D = {D, B, E, F, A, C} using ($AB \rightarrow C$, $D \rightarrow BE$, $E \rightarrow F$, $F \rightarrow A$)

Closure of E = {E,F,A} using ($A \rightarrow F$, $F \rightarrow A$)

Closure of F = {F,A} using($F \rightarrow A$)

Since {C, D} are capable of finding all the attributes so it will be a set of candidate keys. {C, D} will not be part of the next combinations.

Closure of AB = { A, B, C, D, E, F}

Closure of AE = { A,E,F}

Closure of AF = { A,F}

Closure of BE = { A, B, C, D, E, F}

Closure of BF = { A, B, C, D, E, F}

Closure of EF = { A,E,F}

So AB, BE, and BF will be the next candidate keys, and the next other combinations will contain no attributes apart from these candidate keys.

So there will be five candidate keys which are {C, D, AB, BE, BF}

Example 6:

Let's suppose we have a set of attributes as S: {A, B, C, D, E, F, G, H} and functional dependencies are:

A -> BC

E -> A

B -> CFH

CH -> G

F -> EG

Solution:

In the above example, we have non-essential attributes as {A, B, C, E, F, G, H}, and the essential attribute is {D}.

Closure of D = {D}

Since D is an essential attribute, it will be part of all the combinations of essential and non-essential attributes.

Closure of AD = {A, B, C, D, E, F, G, H} using (A->BC, B->CFH, F->EG)

Closure of BD = {A, B, C, D, E, F, G, H} using (B->CFH, F->EG, E->A)

Closure of CD = {C,D}

Closure of ED = {A, B, C, D, E, F, G, H} using (E->A, A->BC, B-> CFH, F->EG)

Closure of FD = {A, B, C, D, E, F, G, H} using (F->EG, E->A, A->BC)

Closure of GD = {D,G}

Closure of HD = {D,H}

Since {AD, BD, ED, FD} are capable of finding all the attributes, there is a set of candidate keys, and {A, B, E, F} will not be part of the next combinations.

Closure CDG = {C, D, G}

Closure CDH = {C, D, H, G}

Closure CDGH = {C, D, G, H}

Now no other combinations are possible because the next combinations will contain the attributes from the set of predetermined candidate keys. Then it will be a super key, not a candidate key.

So, we have tried all the combinations of essential and non-essential attributes, and we got four candidate keys which are {AD, BD, ED, FD}

Dependency Preserving [Refer from PPT]

Problem:

Let a relation R (A, B, C, D) and functional dependency

{AB → C, C → D, D → A}. Relation R is decomposed into R1 (A, B, C) and R2(C, D). Check whether decomposition is dependency preserving or not.

Solution:

R1(A, B, C) and R2(C, D)

Let us find closure of F1 and F2

To find closure of F1, consider all combination of ABC. i.e., find closure of A, B, C, AB, BC and AC
Note ABC is not considered as it is always ABC

closure(A) = { A } // Trivial

closure(B) = { B } // Trivial

closure(C) = {C, A, D} but D can't be in closure as D is not present R1.
= {C, A}

C → A // Removing C from right side as it is trivial attribute

$\text{closure}(AB) = \{A, B, C, D\}$

$= \{A, B, C\}$

$AB \twoheadrightarrow C$ // Removing AB from right side as these are trivial attributes

$\text{closure}(BC) = \{B, C, D, A\}$

$= \{A, B, C\}$

$BC \twoheadrightarrow A$ // Removing BC from right side as these are trivial attributes

$\text{closure}(AC) = \{A, C, D\}$

NULL SET

$F1 \{C \twoheadrightarrow A, AB \twoheadrightarrow C, BC \twoheadrightarrow A\}.$

Similarly $F2 \{C \twoheadrightarrow D\}$

In the original Relation Dependency $\{AB \twoheadrightarrow C, C \twoheadrightarrow D, D \twoheadrightarrow A\}.$

$AB \twoheadrightarrow C$ is present in $F1$.

$C \twoheadrightarrow D$ is present in $F2$.

$D \twoheadrightarrow A$ is not preserved.

$F1 \cup F2$ is a subset of F . So given decomposition is not dependency preserving.